

Bits & Math

JavaScript lo ఒక సంఖ్య 2^{53} వరకు ఖచ్చితం. కానీ `&`, `|`, `>>` వాడిన క్షణం అది 2^{31} కి కుంచింతుకుపోతుంది – మరియు ఎవరూ మీకు చెప్పరు.

మొదటిది – binary search యొక్క `(lo + hi) >> 1` 100 కోట్ల వరకు **0.0%** తప్పు, ఆపై **87.3%**, మరియు అది **ఋణాత్మక** మధ్య బిందువు ఇస్తుంది. రెండోది – `h %` **16** అడిగితే **31 వేర్వేరు buckets** వస్తాయి, మరియు **35.4%** keys ఒక కనిపించని చోట పడతాయి. మూడోది – `n(n+1)/2` **13.4** కోట్ల దగ్గరే అబద్ధం చెప్పడం మొదలుపెడుతుంది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి సంఖ్యా నిజంగా `node lo run చేసినదే` – మరియు ఈ doc lo నా fuzz రెండుసార్లు నా మీదే తిరిగింది: ఒకసారి ఒక నిజమైన ఖాళీ కనిపెట్టి, ఒకసారి నా సూచనే తప్పని చూపించి (§13).

ఈ DOC ఒక్క వాక్యంలో

JavaScript lo రెండు వేర్వేరు సంఖ్యా పరిధులు ఉన్నాయి, మరియు అవి ఒకే విధంగా కనిపిస్తాయి.

అంకగణితం (+ - × / %) – ఖచ్చితం $2^{53} - 1$ వరకు ($\approx 9 \times 10^{15}$).

Bitwise (& | ^ << >> >>> ~) – ఖచ్చితం $2^{31} - 1$ వరకు ($\approx 2 \times 10^9$).

మధ్యలో 40 లక్షల రెట్లు తేడా. మరియు ఆ మధ్య ప్రాంతంలో మీ code crash కాదు – అది నిశ్శబ్దంగా తప్పు జవాబు ఇస్తుంది.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. రెండు పరిధులు, ఒకే రూపం
2. అడగాల్సిన ప్రశ్నలు

Part 2 – మొదటి విరుపు: 32 bits

3. Step – binary search యొక్క మధ్య బిందువు
4. మొదటి విరుపు – 0.0%, ఆపై 87.3%
5. Step – రెండు పరిధులని విడదీయడం

Part 3 – రెండో విరుపు: %

6. Step – hash ని bucket గా మార్చడం
7. రెండో విరుపు – 16 అడిగితే 31 వచ్చాయి
8. Step – కనిపించని గడి

Part 4 – మూడో విరుపు: ఖచ్చితత్వం

9. Step – $n(n+1)/2$
10. మూడో విరుపు – ఖచ్చితత్వం మీరు అనుకున్న చోట ఆగదు

Part 5 – నిరూపణ మరియు ప్రదర్శన

11. మంచి వార్త – Kernighan యొక్క ఉపాయం
12. మొత్తం code
13. 36,000 cases · mutation testing

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. రెండు పరిధులు, ఒకే రూపం

JavaScript lo bitwise ఆపరేషన్లు – సంఖ్యలు 32-bit *సంజ్ఞా* (signed) పూర్ణాంకాలుగా మారతాయి

n	n 0 సరైనదా
-----+-----	
2 ³⁰ = 1,07,37,41,824	1073741824 అవును
2 ³¹ = 2,14,74,83,648	-2147483648 కాదు
2 ³² = 4,29,49,67,296	0 కాదు
2 ⁵³ = 9,00,71,99,25,47,40,992	0 కాదు

మొదటిసారి తప్పుయ్యేది : 2³¹ = 2,14,74,83,648

Number.MAX_SAFE_INTEGER : 9,00,71,99,25,47,40,991 (2⁵³ - 1)

2³¹ | 0 = -2147483648 – ఒక ధనాత్మక సంఖ్య ఋణాత్మకం అయింది

| 0 అనేది "ఏమీ చేయదు" అని చాలామంది అనుకుంటారు – "సున్నాతో OR, అంటే అదే సంఖ్య."

కాదు. | ఆపరేటర్ ముందు సంఖ్యని 32-bit signed పూర్ణాంకంగా మారుస్తుంది, అపై OR చేస్తుంది. ఆ మార్పు కోసేయడం (truncation), మరియు 32వ bit సంజ్ఞ (sign) అవుతుంది.

మరియు అదే సంఖ్య + 1 కి పరిపూర్ణంగా పనిచేస్తుంది. 2³¹ + 1 = 2147483649, ఖచ్చితం.

కాబట్టి "నా సంఖ్యలు ఎంత పెద్దవి?" అనే ప్రశ్నకి జవాబు మరు ఏ ఆపరేటర్ వాడుతున్నారో దాన్ని బట్టి మారుతుంది.

2. అడగాల్సిన ప్రశ్నలు

ప్రశ్న	ఎందుకు అడుగుతున్నాం
సంఖ్యలు 2 ³¹ దాటొచ్చా?	§4 – దాటితే >>, &, అన్నీ అబద్ధం. 2 ³¹ ≈ 214 కోట్లు
విలువలు ఋణాత్మకం కావచ్చా?	§7 – % ఋణాత్మక జవాబు ఇస్తుంది, మరియు array index ఋణాత్మకమైతే crash కాదు

ప్రశ్న	ఎందుకు అడుగుతున్నాం
మొత్తాలు (sums) ఎంత పెద్దవి అవుతాయి?	$\sum 10 - \frac{n(n+1)}{2}$ 13.4 కోట్ల n దగ్గరే పరిధి దాటుతుంది
డబ్బు లెక్కలా?	$\sum 10$ - అయితే float వద్దు. పైసల్లో పూర్ణాంకాలు
ID లు / timestamps ఎంత?	ms timestamps $\approx 1.7 \times 10^{12}$ - 2^31 కంటే 800 రెట్లు పెద్దవి
Hash విలువలు ఏ పరిధిలో?	$\sum 7 - h * 31 + c \mid 0$ ఋణాత్మకం ఇస్తుంది, దాదాపు సగం సార్లు

ఒక ప్రశ్న అడిగితే మరు దీన్ని నిజంగా నడిపారని తెలుస్తుంది

"ఈ సంఖ్యలు 214 కోట్లు దాటొచ్చా?"

ఎందుకంటే జవాబు ఆశ్చర్యకరంగా తరచుగా "అవును":

ms timestamps $\approx 1.7 \times 10^{12} \cdot$ **byte counts** - 3 GB అంటే $3.2 \times 10^9 \cdot$ **user IDs** పెద్ద systems lo $\cdot$ ఏ రెండు పెద్ద సంఖ్యల మొత్తమైనా - $\sum 4$ lo binary search lo అదే జరుగుతుంది.

మరియు 214 కోట్ల లోపల అంతా **0.0% తప్పు**. కాబట్టి మీ tests ఏమీ చెప్పవు.

Part 2 – మొదటి విరుపు: 32 bits

3. Step – binary search యొక్క మధ్య బిందువు

Deep 01 |o మనం రాసినది:

```
const mid = (lo + hi) >> 1;
```

>> 1 అంటే "రెండుతో భాగించు", మరియు అది `Math.floor((lo+hi)/2)` కంటే వేగం. పాఠ్యపుస్తకాలు ఇదే చెబుతాయి, మరియు ఇది పనిచేస్తుంది.

4. మొదటి విరుపు – 0.0%, అపై 87.3%

`|0` ఎన్ని సార్లు తప్పు? · 1,00,000 యాదృచ్ఛిక పూర్ణాంకాలు

పరిధి	తప్పు	%
0 – 10 లక్షలు	0	0.0%
0 – 100 కోట్లు (10^9)	0	0.0%
0 – 10^{10}	78,623	78.6%
0 – 2^{53}	1,00,000	100.0%

100 కోట్ల వరకు సరిగ్గా సున్నా తప్పులు. ఎందుకంటే $2^{31} \approx 214$ కోట్లు, మరియు 100 కోట్లు దాని లోపలే.

binary search యొక్క `(lo + hi) >> 1`

lo	hi	(lo+hi)>>1	lo+(hi-lo)/2	సరైనదా
0	100	50	50	అవును
0	1073741824	536870912	536870912	అవును
1073741824	2147483648	-536870912	1610612736	కాదు
1000000000	2000000000	-647483648	1500000000	కాదు
0	1099511627776	0	549755813888	కాదు

1,00,000 యాదృచ్ఛిక (lo, hi) జతలు · 0–300 కోట్ల పరిధి : 87,306 తప్పు (87.3%)

`MID(109, 2×109) = -647483648`

lo మరియు hi రెండూ 2^{31} కంటే చిన్నవే. ఒక్కొక్కటి సురక్షితమే.

కానీ వాటి మొత్తం 300 కోట్లు – అది 2^{31} దాటుతుంది, మరియు `>>` దాన్ని కోసేస్తుంది.

ఫలితం: ఒక బునాత్మక index. Binary search `arr[-647483648]` చదువుతుంది → `undefined` → పోలిక విఫలం → లేదా అనంత loop.

మరియు 0-100 కోట్ల పరిధిలో ఇది 0.0% తప్పు. మీ tests ఎప్పటికీ చూడవు.

ఇది JAVASCRIPT సమస్య కాదు – ఇది ఒక ప్రసిద్ధ సమస్య

Java యొక్క `Arrays.binarySearch` lo సరిగ్గా ఇదే bug తొమ్మిదేళ్ళు ఉంది, మరియు 2006 lo బయటపడింది. C, C++, Python – ప్రతి భాషలో స్థిర-పరిమాణ పూర్ణాంకాలు ఉన్న చోట.

పరిష్కారం అదే, ప్రతిచోటా: `lo + (hi - lo) / 2`.

`hi - lo` ఎప్పుడూ $\leq hi$, కాబట్టి అది ఎప్పుడూ పరిధి దాటదు.

5. Step – రెండు పరిధులని విడదీయడం

```
// రెండు కారణాలు: (1) lo + hi  $2^{31}$  దాటొచ్చు (2) >> అది 32-bit కి కోసేస్తుంది
function mid(lo, hi) {
    return lo + Math.floor((hi - lo) / 2);
}
// ఈ సంఖ్య మీద bitwise ఆపరేషన్లు సురక్షితమా? – ఊహాని ఒక పంక్తిగా
function bitSafe(n) {
    return Number.isInteger(n) && n >= -(2 ** 31) && n <= 2 ** 31 - 1;
}
```

`>> 1` వదిలేయడం వల్ల ఎంత నెమ్మది?

ఆచరణలో కొలవగలిగేంత కాదు. ఆధునిక JS engines `Math.floor(x / 2)` ని ఒక shift గానే compile చేస్తాయి, మరియు binary search యొక్క ఖరీదు memory చదవడంలో ఉంది, ఈ లెక్కలో కాదు.

కాబట్టి ఇది ఒక trade-off కాదు. `>> 1` వేగం కోసం కాదు – అది అలవాటు.

మరియు §12 lo mid ని `(lo + hi) >> 1` గా మార్చితే 120/120 విఫలం.

Part 3 – రెండో విరుపు: %

6. Step – hash ని bucket గా మార్చడం

```
function hash(s) { let h = 0; for (const c of s) h = (h * 31 + c.charCodeAt(0)) | 0;
return h; }

const BUCKETS = 16;
const buckets = new Array(BUCKETS).fill(null).map(() => []);
buckets[hash(key) % BUCKETS].push(key);
```

ఇది పాఠ్యపుస్తక hash map. % BUCKETS విలువని 0 నుంచి 15 కి తెస్తుంది.

లేదా తెస్తుందా?

7. రెండో విరుపు – 16 అడిగితే 31 వచ్చాయి

`%` – JavaScript లో అది "మిగులు" (remainder), "modulo" కాదు

a	b	a % b	నిజమైన mod
7	3	1	1
-7	3	-1	2
-1	10	-1	9
-13	5	-3	2
-100	7	-2	5

2,00,000 keys · 16 buckets

h % 16 ఋణాత్మకం వచ్చినవి : 70,845 (35.4%)
h % 16 ఇచ్చిన వేర్వేరు విలువలు : 31 → -15 -14 -13 ... 13 14 15
(h % 16) + 16 ఇచ్చినవి : 16 → 0..15

16 BUCKETS అడిగాను. 31 వచ్చాయి.

hash() lo | 0 ఉంది (§1), కాబట్టి అది **ఋణాత్మక** విలువలు ఇస్తుంది – దాదాపు సగం సార్లు.

మరియు $-7 \% 3$ JavaScript lo **-1, 2** కాదు. C, Java, Go – అన్నీ అంతే. Python మాత్రం **2** ఇస్తుంది.

కాబట్టి $h \% 16$ యొక్క పరిధి **[0, 15]** కాదు – అది **[-15, 15]**, అంటే **31 విలువలు**.

మరియు **35.4% keys ఋణాత్మక index కి వెళ్ళాయి**.

8. Step – కనిపించని గడి

ఋణాత్మక index తో ఏమవుతుంది? **ఏమీ కాదు**. అదే సమస్య.

```
const arr = new Array(4).fill(0);
arr[-3] = 99;
```

```
arr      → [0, 0, 0, 0]
arr.length → 4
arr[-3]   → 99
```

JAVASCRIPT LO ARRAY INDEX ఒక STRING PROPERTY. -3 కూడా ఒక చెల్లుబాటు అయ్యే పేరు.

`arr[-3] = 99` crash కాదు, హెచ్చరించదు, `.length` మార్చదు.

అది ఆ object మీద `"-3"` అనే ఒక **property** సృష్టిస్తుంది. Array యొక్క *index* భాగంలో అది లేదు.

కాబట్టి `for (let i = 0; i < arr.length; i++)` దాన్ని **ఎప్పటికీ చూడదు**. `JSON.stringify` దాన్ని **వదిలేస్తుంది**. `arr.map`, `arr.filter`, `for...of` – ఏదీ చూడదు.

70,845 keys ఒక కనిపించని చోట పడ్డాయి, మరియు ఒక్క error కూడా రాలేదు.

పరిష్కారం మూడు అక్షరాలు:

```
function mod(a, b) {
  return ((a % b) + b) % b;    // ఫలితం ఎప్పుడూ [0, b)
}
```

ఆ **చివరి** $\% b$ కూడా అవసరమే – a ధనాత్మకమైతే $(a \% b) + b$ అనేది b దాటుతుంది. §13 lo దాన్ని తీసేస్తే **120/120** విఫలం.

Part 4 – మూడో విరుపు: ఖచ్చితత్వం

9. Step – $n(n+1)/2$

```
const sum = n * (n + 1) / 2;    // loop కంటే అనంతంగా వేగం
```

Gauss యొక్క సూత్రం. $n = 1000$ ✓ · $n = 10^6$ ✓ · $n = 10^7$ ✓.

10. మూడో విరుపు – ఖచ్చితత్వం మీరు అనుకున్న చోట ఆగదు

1 నుంచి n వరకు మొత్తం

n	$n(n+1)/2$	loop తో కూడినది	సరైనదా
1000	500500	500500	అవును
1000000	500000500000	500000500000	అవును
10000000	50000005000000	50000005000000	అవును
150000000	11250000075000000	11250000067108864	కాదు

$n(n+1)/2 > \text{MAX_SAFE_INTEGER}$ అయ్యే మొదటి 2 ఘాతం : $n = 2^{27} = 13,42,17,728$

తేడా 79,89,136 – మరియు ఏ ERROR, ఏ హెచ్చరిక లేదు

$n = 15$ కోట్లు. ఇది ఒక పెద్ద సంఖ్య కానీ అసాధ్యమైనది కాదు – log rows, event counts, byte offsets.

n ఖచ్చితం. $n + 1$ ఖచ్చితం. కానీ **ఫలితం** 1.1×10^{16} – అది 2^{53} దాటుతుంది, మరియు అక్కడి నుంచి doubles **ప్రతి పూర్ణాంకాన్నీ** పట్టలేవు.

కాబట్టి జవాబు **అత్యంత దగ్గరి** double కి గుండ్రంగా మారుతుంది. అది ఇంకా ఒక పూర్ణాంకంలాగే కనిపిస్తుంది, `Number.isInteger` కి `true` అంటుంది, మరియు అది **తప్పు**.

దశాంశ అంకగణితం

```
0.1 + 0.2 === 0.3 ? false    (0.1 + 0.2 = 0.30000000000000004)
1,00,000 జతల రెండు-దశాంశ సంఖ్యలు · a + b ఖచ్చితం కానివి : 22,781 (22.8%)
```

22.8% - ఇది "అరుదైన FLOATING POINT విచిత్రం" కాదు

రెండు దశాంశ స్థానాల సంఖ్యలు అంటే రూపాయలు మరియు పైసలు. ప్రతి నాలుగో కూడిక ఖచ్చితం కాదు.

ఒక్కో తేడా 10^{-16} - చిన్నదే. కానీ అవి పొగవుతాయి, మరియు `===` తో పోల్చినప్పుడు తేడా ఎంత చిన్నదైనా అది తప్పే.

పరిష్కారం: డబ్బుని పైసల్లో పూర్ణాంకాలుగా ఉంచండి. ₹12.30 కాదు - **1230**. కూడికలు, తీసివేతలు ఖచ్చితం, మరియు చూపించేటప్పుడు మాత్రమే 100 తో భాగించండి.

Part 5 – నిరూపణ మరియు ప్రదర్శన

11. మంచి వార్త – Kernighan యొక్క ఉపాయం

ఈ doc మొత్తం "bitwise ప్రమాదం" అని చెబుతోంది. కాబట్టి నిజంగా బాగా పనిచేసేది కూడా కొలుద్దాం.

$n \& (n - 1)$ – అతి కింది set bit ని తీసేస్తుంది. కాబట్టి set bits లెక్కపెట్టడానికి 32 అడుగులు అక్కర్లేదు, ఉన్నన్ని చాలు.

2^{20} సంఖ్యలకి Kernighan అడుగులు

ఆకారం	అడుగులు	ఒక్కేదానికి సగటు
యాదృచ్ఛికం	1,62,57,272	15.50
2 ఘాతాలు (ఒక్క bit)	10,48,576	1.00
అన్నీ 1 ($2^k - 1$)	1,57,12,438	14.98

(naive రూపం ఎప్పుడూ 32 అడుగులు.)

15.50 vs 32 – సగం. మరియు 2 ఘాతాలకి **1.00**, అంటే ఒకే అడుగు.

కానీ 32 bits దాటితే –

n	Kernighan	నిజం	సరైనదా
255	8	8	అవును
2^{35}	1	1	అవును ← *ప్రమాదవశాత్తు*
$2^{35} + 2^{36}$	1	2	కాదు
$2^{40} - 1$	32	40	కాదు
MAX_SAFE_INTEGER	32	53	కాదు

మొదటిసారి తప్పుయ్యేది : $2^{32} + 1$

2^{35} దగ్గర జవాబు సరైనది – కానీ సరైన కారణంతో కాదు

2^{35} ని 32-bit కి కోస్ట్ 0 వస్తుంది. Loop మొదటి అడుగులోనే $c = 1$ చేసి ఆగిపోతుంది. నిజమైన జవాబు కూడా 1. యాదృచ్ఛికంగా సరిపోయింది.

$2^{35} + 2^{36}$ దగ్గర అదే తర్కం 1 ఇస్తుంది, మరియు నిజం 2.

"నా పరీక్షలు నెగ్గాయి" అనేది "నా code సరైనది" కాదు. కొన్నిసార్లు అది "నా పరీక్షలు అదృష్టవంతమైనవి".

పరిష్కారం: సంఖ్యని రెండు 32-bit ముక్కలుగా విడదీసి, ఒక్కో ముక్క మీద Kernighan.

12. మొత్తం code

```
// set bits ఎన్ని · పూర్తి safe-integer పరిధిలో
// Kernighan యొక్క `n &= n - 1` 32 bits కీ. కాబట్టి సంఖ్యని రెండు ముక్కలుగా విడదీస్తాం.
function popcount(n) {
  if (!Number.isInteger(n) || n < 0) return -1;
  let lo = n % 4294967296;           // కింది 32 bits
  let hi = Math.floor(n / 4294967296); // పైవి
  let c = 0;
  for (const part of [lo, hi]) {
    let x = part >>> 0;           // unsigned గా చూడటం
    while (x) { x &= x - 1; c++; }
  }
  return c;
}
```

```
// 1 నుంచి n వరకు మొత్తం · సూత్రం సురక్షితమైతే మాత్రమే
function sumTo(n) {
  if (!Number.isInteger(n) || n < 0) return NaN;
  const s = n * (n + 1) / 2;
  return Number.isSafeInteger(s) ? s : NaN; // అబద్ధం చెప్పే కంటే NaN మేలు
}
```

ఆ చివరి పంక్తి ఒక రూపకల్పన నిర్ణయం

`sumTo(15 కోట్లు)` కి మూడు ఎంపికలు: **(1)** తప్పు సంఖ్య ఇవ్వడం · **(2)** NaN ఇవ్వడం · **(3)** BigInt కి మారడం.

మొదటిది ఎప్పుడూ తప్పు – §10 యొక్క 79,89,136 తేడా నిశ్శబ్దంగా ముందుకి ప్రయాణిస్తుంది.

నేను రెండేది ఎంచుకున్నాను: తెలియనప్పుడు "తెలియదు" అనడం. అదే Deep 10 lo LCA కి null ఇవ్వడం, మరియు Deep 11 lo -1 కి బదులు index ఇవ్వడం.

ఒక function యొక్క అత్యంత ముఖ్యమైన సామర్థ్యం: "నాకు తెలియదు" అని చెప్పగలగడం.

13. 36,000 cases · mutation testing

పదిహేడు నియమాలు: `mod` ఎప్పుడూ `[0, b)`, `mod = BigInt` సూచన, `(a - mod) b` తో భాగించబడాలి, `mid ∈ [lo, hi]`, `mid = BigInt` సూచన, `mid` ముందుకి కదలాలి, `popcount = BigInt` సూచన, `lowestSetBit = BigInt` సూచన, అది `n` ని భాగించాలి, అది 2 ఘాతం కావాలి, `isPowerOfTwo` ⇔ `popcount = 1`, `bitSafe` నిజంగా `(n\|0)===n` నే చెప్పాలి, `gcd ≥ 0` మరియు సూచనతో సరిపోవాలి, `gcd` రెండింటినీ భాగించాలి, `gcd × lcm = |a×b|`, `sumTo NaN` ఇస్తే అది నిజంగా అసురక్షితమే కావాలి, `sumTo` సంఖ్య ఇస్తే అది BigInt తో ఖచ్చితంగా సరిపోవాలి.

ప్రతి సూచనా BigInt తో రాసినది – కాబట్టి అవి 32-bit ఉచ్చులో పడవు.

300 runs · 9,000 mod · 9,000 mid · 9,000 popcount · 6,000 gcd

నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

mods	9,000	
negA	4,470	← ఋణాత్మక a · §7 యొక్క దారి
mids	9,000	
bigMid	4,452	← lo + hi 2 ³¹ దాటినవి · §4 యొక్క దారి
pops	9,000	
bigPop	4,884	← 32 bits దాటిన సంఖ్యలు
gcds	6,000	
negGcd	4,524	
sums	3,000	
unsafeSum	638	← సూత్రం పరిధి దాటినవి

==== mutation testing ====

మార్పు లేని code	→ 0/300 విఫలం
mod lo `((a%b)+b)%b` ని `a%b` గా (§7 యొక్క bug)	→ 120/120 విఫలం
ఉదా: mod(-853410044, 704) = -124, [0, 704] బయట	
mod lo చివరి `% b` తీసేస్తే	→ 120/120 విఫలం
ఉదా: mod(805712038, 971) = 1484, [0, 971] బయట	
mid lo `lo + (hi-lo)/2` ని `(lo + hi) >> 1` గా (§4)	→ 120/120 విఫలం
ఉదా: mid(2718898685649037, 3336033055558800) = -741771634, [lo, hi] బయట	
mid lo `Math.floor` ని `Math.ceil` గా	→ 120/120 విఫలం
popcount lo ముక్కలుగా విడదీయకుండా నేరుగా Kernighan	→ 120/120 విఫలం
ఉదా: popcount(68719476735) = 32, సూచన 36	
popcount lo `part >>> 0` ని `part   0` గా	→ 0/120 విఫలం
lowestSetBit lo `x & -x` ని `x & (x-1)` గా	→ 120/120 విఫలం
lowestSetBit lo పై ముక్కని 2 ³² తో గుణించకపోతే	→ 113/120 విఫలం
gcd lo `Math.abs` తీసేస్తే	→ 120/120 విఫలం
ఉదా: gcd(-279873, 41665) = -1 - ఋణాత్మకం	
sumTo lo `Number.isSafeInteger` తనిఖీ తీసేస్తే	→ 113/120 విఫలం
ఉదా: sumTo(417980725) = 87353943444753170 - ఇది safe integer కాదు	
bitSafe lo హద్దుని 2 ³¹ కి బదులు 2 ⁵³ గా	→ 120/120 విఫలం

SUMTO MUTATION మొదట 0/120 గా బతికింది - మరియు కారణం నా సూచన

నా నియమం ఇలా ఉండేది: sumTo(n) ని Number(BigInt(n) * BigInt(n+1) / 2n) తో పోల్చడం.

BigInt భాగం ఖచ్చితమే. కానీ ఆ చివరి Number(...) ఫలితాన్ని తిరిగి ఒక double కి మారుస్తుంది - మరియు అది సరిగ్గా అదే గుండ్రం చేస్తుంది.

కొలిచాను: సురక్షితం కాని 2,19,094 సందర్భాల్లో $n*(n+1)/2$ మరియు Number(BigInt...) ఒక్కసారి వేరుగా రాలేదు.

నా సూచన పరీక్షించబడుతున్న code తో ఒక తప్పు చేస్తోంది. ఇది Deep 08 యొక్క వృత్తాకార నియమమే, కొత్త రూపంలో.

పరిష్కారం: పోలికని Number గుండా కాకుండా BigInt(s) !== wantBig అని BigInt lo చేయడం. → 0/120 నుంచి 113/120.

దీన్ని కూడా సరిచేయడానికి ప్రయత్నించాను, మరియు సరిచేయలేకపోయాను – ఎందుకంటే ఈ మార్పు ఏమీ మార్చదు.

>>> 0 మరియు | 0 రెండూ ఒకే 32 bits ఇస్తాయి; తేడా కేవలం ఆ bits ని సంజ్ఞాతో చదువుతామా లేదా అనేది. తర్వాతి `x &= x - 1` మళ్ళీ 32-bit ఆపరేషన్, మరియు `while (x)` కి సున్నా కానిదంతా ఒకటే.

కొలిచాను: 20,00,000 విలువల మీద 0 తేడాలు.

ఇది ఈ series lo ఆరో రకం: బలహీన పరీక్ష · చచ్చిన code · harness చేరుకోలేనిది · సరిపోని data · వృత్తాకార పరీక్ష · మరియు ఇప్పుడు – సమానార్థక mutation (equivalent mutant). ఆ మార్పు ఒక bug కాదు; >>> 0 అక్కడ స్పష్టత కోసం, సరిదిద్దుబాటు కోసం కాదు.

14. Follow-ups – BigInt, typed arrays, bitsets

"2^53 దాటాలంటే?"

BigInt – పరిమితి లేని పూర్ణాంకాలు. `10n ** 100n` పనిచేస్తుంది. ఖరీదు: **Number** తో కలపలేము (`1n + 1` ఒక **TypeError**), మరియు అంకగణితం గణనీయంగా నెమ్మది. ఈ doc యొక్క సూచనలన్నీ BigInt తోనే రాశాను – కానీ దాని వేగాన్ని కొలవలేదు.

"32 bits మీద నిజంగా పని చేయాలంటే?"

Typed arrays – `Int32Array`, `Uint32Array`. ఇవి ప్రకటించబడిన 32-bit, కాబట్టి కోసేయడం ఒక ఆశ్చర్యం కాదు, అది ఒప్పందం. Hash tables, image buffers, checksums కి ఇదే సరైన సాధనం.

"లక్షల boolean flags కి?"

Bitset – `Uint32Array` lo ఒక్కో bit ఒక flag. 10 లక్షల flags = **1,25,000 bytes**, ఒక `Array` lo అయితే **80 లక్షల bytes**. `set(i)` అంటే `a[i >> 5] |= 1 << (i & 31)`. నేను bitsets ని కొలవలేదు.

"డబ్బు లెక్కలకి library ఉందా?"

`decimal.js`, `big.js` – దశాంశ అంకగణితాన్ని ఖచ్చితంగా చేస్తాయి. కానీ §10 lo చూసినట్టు, పైసల్లో పూర్ణాంకాలు చాలా సందర్భాల్లో సరిపోతాయి మరియు ఉచితం. నేను ఈ libraries ని కొలవలేదు.

"ఈ doc lo నేను ఏమి కొలవలేదు"

నాలుగు: **BigInt** యొక్క వేగం, **typed arrays**, **bitsets**, మరియు **decimal libraries**.

15. ఏమి నేర్చుకున్నాం

1. JavaScript lo రెండు సంఖ్య పరిధులు ఉన్నాయి. అంకగణితం 2^{53} , bitwise 2^{31} . మధ్యలో 40 లక్షల రెట్లు.
2. `| 0` "ఏమీ చేయదు" కాదు – అది 32-bit కి కోసేస్తుంది. $2^{31} | 0 = -2147483648$.
3. `(lo + hi) >> 1` 100 కోట్ల వరకు 0.0% తప్పు, 300 కోట్ల దగ్గర 87.3% – మరియు అది ఋణాత్మక index ఇస్తుంది. `lo + (hi - lo) / 2` రాయండి.
4. ఇది Java `Arrays.binarySearch` lo తొమ్మిదేళ్ళు ఉంది. మీ code lo ఉండటం ఆశ్చర్యం కాదు.
5. `%` "మిగులు", "modulo" కాదు. `-7 % 3 = -1`. `((a % b) + b) % b` రాయండి – చివరి `% b` తో సహా.
6. 16 buckets అడిగితే 31 వచ్చాయి, మరియు 35.4% keys ఋణాత్మక index కి వెళ్ళాయి.
7. `arr[-3] = 99` crash కాదు, `.length` మార్చాడు, మరియు ఏ loop దాన్ని చూడదు. నిశ్శబ్ద data నష్టం.
8. `n(n+1)/2` 13.4 కోట్ల దగ్గరే పరిధి దాటుతుంది – తేడా 79,89,136, error సున్నా.
9. రెండు-దశాంశ కూడికల్లో 22.8% ఖచ్చితం కాదు. డబ్బుని ఫైసల్ పూర్ణాంకాలుగా ఉంచండి.
10. "నాకు తెలియదు" అని చెప్పగలగడం ఒక సామర్థ్యం. `sumTo` తప్పు సంఖ్య కంటే NaN ఇస్తుంది.
11. నా సూచన తప్పు చేసింది – BigInt ఫలితాన్ని `Number()` గుండా పంపడం వల్ల అది పరీక్షించబడుతున్న code తోనే ఒకే తప్పు చేసింది. `0/120 → 113/120`.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

ఒక ఆపరేటర్ మీ సంఖ్య ఎంత పెద్దదో మార్చగలడు, మరియు అది మీకు చెప్పదు.

`n + 1` – 2^{53} వరకు సరైనది. `n | 0` – 2^{31} వరకు. `n * (n+1) / 2` – 2^{53} ఫలితం వరకు. `a % b` – `a` ధనాత్మకమైతే మాత్రమే మీరు అనుకున్నది.

నాలుగూ ఒకేలా కనిపిస్తాయి, నాలుగూ crash కావు, మరియు నాలుగింటికీ మీ చిన్న test data ఏమీ చెప్పదు.

కాబట్టి ఒక్క ప్రశ్న: "ఈ సంఖ్య 214 కోట్లు దాటొచ్చా? ఋణాత్మకం కావచ్చా?" జవాబు "అవును" లేదా "తెలియదు" అయితే – `mod()` మరియు `mid()` రాయండి.